

Data: Data means value or set of values. In both the Singular and plural form, example

34

09/06/1974

DATA [-2]

DATA (-2)

DATA₋₂

1. $O(n^3)$
2. $O(n^r)$ } polynomial

3. $O(n \log_2 n)$

4. $O(\log_2 n)$

5. $O(2^n)$ → Exponential

trade off → memory or time

$$\lfloor 2.7 \rfloor = 2 \quad \lfloor \cdot \rfloor \Rightarrow \text{Floor}$$

$$\lceil -2.7 \rceil = -2 \quad \lceil \cdot \rceil \Rightarrow \text{Ceiling}$$

$$\lceil 2.7 \rceil = 3$$

$$\text{INT}(\sqrt{5}) = 2$$

$$a_1 + a_2 + a_3 + \dots + a_n$$

$$\sum_{i=1}^n a_i$$

$$a_1 \times a_2 \times a_3 \times \dots \times a_n$$

$$\prod_{i=1}^n a_i$$

$$\frac{250!}{245!6!} = 250 \times 249 \times 248 \times 247 \times 246 \times 245 \times \dots \times 20$$

❖ Algorithm and Flow chart

Step 1: Start

Step 2: Set MAX := DATA[1]

Set I := 2

Step 3: Repeat steps 4 & 5 while $I \leq N$

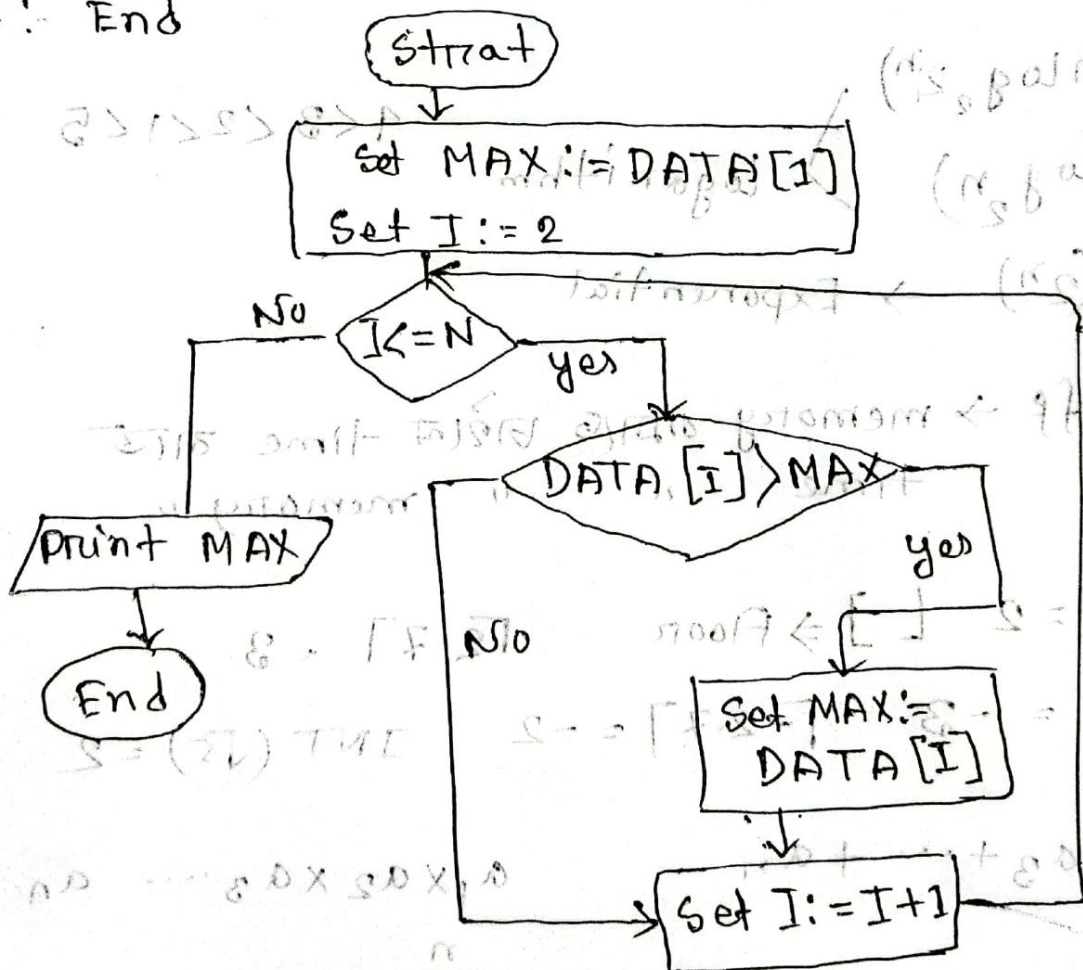
Step 4: If $DATA[I] > MAX$, then

Set $MAX := DATA[I]$

Step 5: Set $I := I + 1$

Step 6: Print MAX

Step 7: End



⑧ Linear search Algorithm

Step 1: Set $k := 1$ and $loc := 0$

Step 2: Repeat while $k \leq N$

Step 3: If $ITEM = DATA[k]$ then set $LOC := k$

Step 4: $k := k + 1$

Step 5: If $LOC := 0$ then

print ITEM Not Found

else

print ITEM found at location

Step 6: Exit

Q Array 4th Element Insert kora

Step 1: Start

1	2	3	4	5	6	7	8
5	2	3	4	7	9	0	

Step 2: Set $I := N$

Step 3: Repeat steps 4 & 5 while $I \geq K$

Step 4: Set $DATA[I+1] := DATA[I]$

Step 5: Set $I := I - 1$

Step 6: Set $DATA[K] := X$

Step 7: END

Q Array 4th Element Delete kora

Step 1: Start

1	2	3	4	5	6	7
10	20	30	40	50	60	70

Step 2: Set $I := K$

Step 3: Repeat steps 4 & 5 while $I \leq N-1$

Step 4: Set $DATA[I] := DATA[I+1]$

Step 5: Set $I := I + 1$

Step 6: END

Memory Address: - 200

15	16	17	18	19	20	21
-2	7	-3	17	50	99	87
200	204	208				

$$LOC(A[K]) = \text{Base}(A) + w(K - \text{Lower Bound})$$

$$\Rightarrow LOC(A[19]) = 200 + 4(19 - 15)$$

$$= 216$$

$$\text{Number of elements} = 15 - 21 + 1 = 7$$

□ BUBBLE_SORT(DATA, N)

Step 1: Repeat step 2 & 3 for $k=1$ to $N-1$

Step 2: Set $PTR := 1$

Step 3: Repeat while $PTR \leq N-k$

(a) If $DATA[PTR] > DATA[PTR+1]$ Then

Swap $DATA[PTR]$ & $DATA[PTR+1]$

(b) Set $PTR := PTR + 1$

Step 4: Exit

set $BEG := MID + 1$

End set $END := MID - 1$

set $MID := INT((BEG + END) / 2)$

Complexity:-

$$(N-1) + (N-2) + (N-3) + (N-4)$$

$$4 + 3 + 2 + 1$$

$$\therefore \frac{n(n+1)}{2} = \frac{n^2 + n}{2}$$

$$= \frac{n^2}{2} + \frac{n}{2}$$

Complexity $O(n^2)$

		1	2	3	4	5
k	Ptr	4	9	5	1	0
1	1	4	9	5	1	0
	2	4	⑤	⑨	1	0
	3	4	5	①	⑨	0
	4	4	5	1	①	⑨
2	1	4	①	⑤	0	9
	2	4	1	①	⑤	9
	3					
3	1	①	⑨	0	5	9
	2	1	①	⑨	5	9
4	1	①	①	4	5	9

Binary search:-

- ① Set $BEG := LB$, $END := UB$, $MID := \text{INT}((BEG + END)/2)$
- ② Repeat steps 1 while $BEG \leq END$ and $DATA[MID] \neq ITEM$
- ③ If $ITEM > DATA[MID]$, then:
 set $BEG := MID + 1$
 Else set $END := MID - 1$
- ④ Set $MID := \text{INT}((BEG + END)/2)$

- 5) If $DATA[MID] = ITEM$, then:
 write "ITEM was found at MID"
 Else
 write "ITEM was not found in DATA"
- 6) Exit.

17 18 19 20 21 22 23 24 25 26 27 28 29

11	22	30	33	40	44	55	60	66	77	80	88	99
----	----	----	----	----	----	----	----	----	----	----	----	----

BEG	END	MID
17	29	23
20	22	19
		21

complexity $O(\log_2 n)$

~~Time complexity :-~~

$\Rightarrow$ int $n=5$

for (int $i=1$; $i \leq n$; $i++$)

for (int $j=1$; $j \leq i$; $j++$) {

cout << "Hello!";

}

}

• complexity $O(n^2)$

$\Rightarrow$ for (int $i=1$; $i \leq \sqrt{n}$; $i++$) {
 cout << "Hello!";
 }

complexity $O(\sqrt{n})$

```

⇒ for (int i=1; i*i <= n; i++) {
    for (int j=1; j <= n; j++) {
    }
}

```

complexity $O(n\sqrt{n})$

```

⇒ int n = 64;
while (n != 0) {
    cout << "Hello";
    n = n/2;
}

```

64 → 32 → 16 → 8 → 4 → 2 → 1

$64 = 2^6$
 $\log_2 64$
 $6 + 7 = 13$

complexity $O(\log_2 n)$

```

⇒ int n = 64;
while (n != 0) {
    print("HI");
    n = n/7;
}

```

complexity $O(\log_7 n)$


```

⇒ int sum(int x, int y, int z) {
    return x+y+z;
}

```

```

int main() {
    int a = sum(1, 9, 2);
    cout << a;
}

```

complexity $O(1)$.

Pattern Matching: $\phi = aab$

T =	a	b	a	b	b	a	b	b	b	a	b	a	a	a	a	b	b	b
	a	a	b	→	2													
		a	a	b	→	1												
			a	a	b	→	2											
				a	a	b	→	1										
					a	a	b	→	1									
						a	a	b	→	2								
							a	a	b	→	1							
								a	a	b	→	1						
									a	a	b	→	2					
										a	a	b	→	1				
											a	a	b	→	3			
												a	a	b	→	3		
													a	a	b	→	3	

$$C = 2 + 1 + 2 + 1 + 1 + 2 + 1 + 1 + 1 + 2 + 1 + 3 + 3 + 3$$

$$= 24$$

$$\text{INDEX}(\tau, \phi) = 14$$

$$\text{complexity } O(mn) \rightarrow O(n^2)$$

Row major order

	7	8	9	10	11	12	13
-2	100						124
-1	128						152
0	156						180
1	184				200	204	208
2							

A

N

$$\text{SIZE} = M \times N$$

$$A[j, k]$$

$$\text{LOC}(A[j, k]) = \text{Base}(A) + w(N * (j - \text{LB}_R) + (k - \text{LB}_C))$$

$$= 100 + 4 \{ 7 * (1 - (-2)) + (11 - 7) \}$$

$$= 100 + 4(21 + 4)$$

$$= 200$$

Column major order:- $\text{Base}(A) + w(M * (K-1) + (J-1))$

⑧ $A(2:8, -4:1, 6:10)$

$$\text{Base}(A) = 200$$

$$w = 4$$

Number of elements?

$$\text{Loc}(A[5, -1, 8]) = ?$$

$$\Rightarrow L_1 = 8 - 2 + 1 = 7$$

$$L_2 = 1 - (-4) + 1 = 6$$

$$L_3 = 10 - 6 + 1 = 5$$

$$\text{Num of elements} = 7 \times 6 \times 5 = 210$$

Now, $E_1 = 5 - 2 = 3$

$$E_2 = -1 - (-4) = 3$$

$$E_3 = 8 - 6 = 2$$

$$\text{Loc}(A[5, -1, 8]) = \text{Base}(A) + w \{ (E_1 L_2 + E_2) L_3 + E_3 \}$$

$$= 200 + 4 \{ (3 \times 6 + 3) 5 + 2 \}$$

$$= 628 \quad \underline{\text{Ans}}$$

STRING :-

S = "TO BE OR NOT TO BE"

T₁ = "TO BE"

T₂ = "NOT BE"

T₃ = "BE"

LENGTH(T₁) = 5

LENGTH(T₂) = 6

LENGTH(" ") = 0

T₁ || T₂ = "TO BE NOT BE"

T₁ || " " || T₂ = "TO BE NOT BE"

SUBSTRING(S, 3, 5) = "BE "

INDEX(S, T₁) = 1

INDEX(S, T₂) = 0

INDEX(S, T₃) = 4

INSERT(T₂, 5, "TO ") = "NOT TO BE"

DELETE(S, 4, 3) = "TO OR NOT TO BE"

{s[i] + s[j] (s.length)} w.r. (A) s.w.s. ([s, i, j] A)

{s[i] s (s + 1 x s)} p + 000 =

3 (A) 882 =

STACK:- LIFO (last input - first output)

$N = 4$

TOP = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~2~~ ~~3~~ ~~2~~ ~~1~~

if TOP + 1 $\leq$ N

5	-2	7 ₁	3
---	----	---------------------------	---

PUSH (5)

PUSH (-2)

PUSH (7)

POP()

✓ PUSH(1)

✓ PUSH(3)

✗ PUSH(9)

✗ PUSH(11)

Stack overflow

✓ POP()

✓ POP()

✓ POP()

✓ POP()

✗ POP()

Stack underflow

$\Rightarrow (A + B) * C \Rightarrow AB + C *$

$\Phi = 5, 6, 2, +, *, 12, 4, /, -$
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10)

Symbol	Stack
(1) 5	5
(2) 6	5, 6
(3) 2	5, 6, 2
(4) +	5, 8
(5) *	40
(6) 12	40, 12

(7) 4	40, 12, 4
(8) /	40 - 3
(9) -	37
(10))	

9

3 1 - 2 ↑ N 4 / 2 * + 5 -) = XY
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14)

Symbol	Stack
① 3	3
② 1	3, 1
③ -	2
④ 2	2, 2
⑤ ↑	4
⑥ N	4, N
⑦ 4	4, N, 4
⑧ /	4, N/4
⑨ 2	4, N/4, 2
⑩ *	4, N/2
⑪ +	4 + N/2
⑫ 5	4 + N/2 - 5
⑬ -	4 + N/2 - 5
⑭)	

$$\therefore \frac{8 + N - 10}{2} = XY = 34$$

$$\text{or, } \frac{N - 2}{2} = 34$$

$$\text{or, } N - 2 = 68$$

$$\therefore N = 70 \quad \underline{\text{Ans}}$$

Infix to postfix :-

Q $A + (B * C - (D / E \uparrow F) * G) * H$

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Symbol	Stack	Postfix exp.
① A	(	A
② +	(+	A
③ (	(+ (	A
④ B	(+ (	AB
⑤ *	(+ (*	AB
⑥ C	(+ (*	ABC
⑦ -	(+ (-	ABC *
⑧ (	(+ (- (	ABC *
⑨ D	(+ (- (	ABC * D
⑩ /	(+ (- (/	ABC * D
⑪ E	(+ (- (/	ABC * DE
⑫ ↑	(+ (- (/ ↑	ABC * DE
⑬ F	(+ (- (/ ↑	ABC * DEF
⑭)	(+ (-	ABC * DEF ↑ /
⑮ *	(+ (- *	ABC * DEF ↑ /
⑯ G	(+ (- * ↑	ABC * DEF ↑ / G
⑰)	(+	ABC * DEF ↑ / G * -
⑱ *	(+ *	ABC * DEF ↑ / G * -
⑲ H	(+ *	ABC * DEF ↑ / G * - H
⑳)	empty	ABC * DEF ↑ / G * - H * +

↑ /
*
+ -
(+ / ↑
⇒ * / ⇒ (+ *
(/ * + * ↑
⇒ * / ⇒ (/ * + *

⑧ TOP = 4

STACK: 227, 123, N, 583, ---, ---, ---, ---

①

* Call POP (STACK, ITEM A)

ITEM A = 583

STACK: 227, 123, N, ---, ---, ---, ---

TOP = 3

* Call POP (STACK, ITEM B)

ITEM B = N

STACK: 227, 123, ---, ---, ---, ---, ---, ---

TOP = 2

* Call POP (STACK, ITEM C)

ITEM C = 123

STACK: 227, ---, ---, ---, ---, ---, ---, ---

TOP = 1

* Call PUSH (STACK, ITEM A + 140)

STACK: 227, 723, ---, ---, ---, ---, ---, ---

TOP = 2

* Call PUSH (STACK, 190)

STACK: 227, 723, 190, ---, ---, ---, ---, ---

TOP = 3

* Call PUSH (STACK, ITEM B + ITEM C)

STACK: 227, 723, 190, N + 123, ---, ---, ---, ---

TOP = 4

⑧ Write an algorithm that receives a bracket sequence and tells whether the seq. is correct or incorrect.

⇒ Step 0: Start

Step 1: Set $k := 1$, $VALID := TRUE$, $STACK$ is empty.

Step 2: Repeat steps 3 to 7 while $k \leq N$

Step 3: If $SEQ[k] = '('$ or $\{$ or $['$ then:

Push $SEQ[k]$ to $STACK$ and go to step 7

Else If $SEQ[k] = ')'$ then:

Go to step 4

Else if $SEQ[k] = '\}'$ then:

Go to step 5

Else if $SEQ[k] = '\}]'$ then

Go to step 6

Step 4: If $STACK$ is non-empty and $STACK[TOP] = '('$,

then: POP from $STACK$ and go to step 7

Else, Set $VALID := FALSE$ and go to step 8

Step 5: If $STACK$ is non empty and $STACK[TOP] = '\{'$,

then: POP from $STACK$ and go to step 7

Else, Set $VALID := FALSE$ and go to step 8

Step 6: If STACK is non-empty and $STACK[TOP] = '['$ then:

Pop from STACK, and go to step 7

Else, set $VALID := FALSE$, and go to step 8

Step 7: Set $k := k + 1$

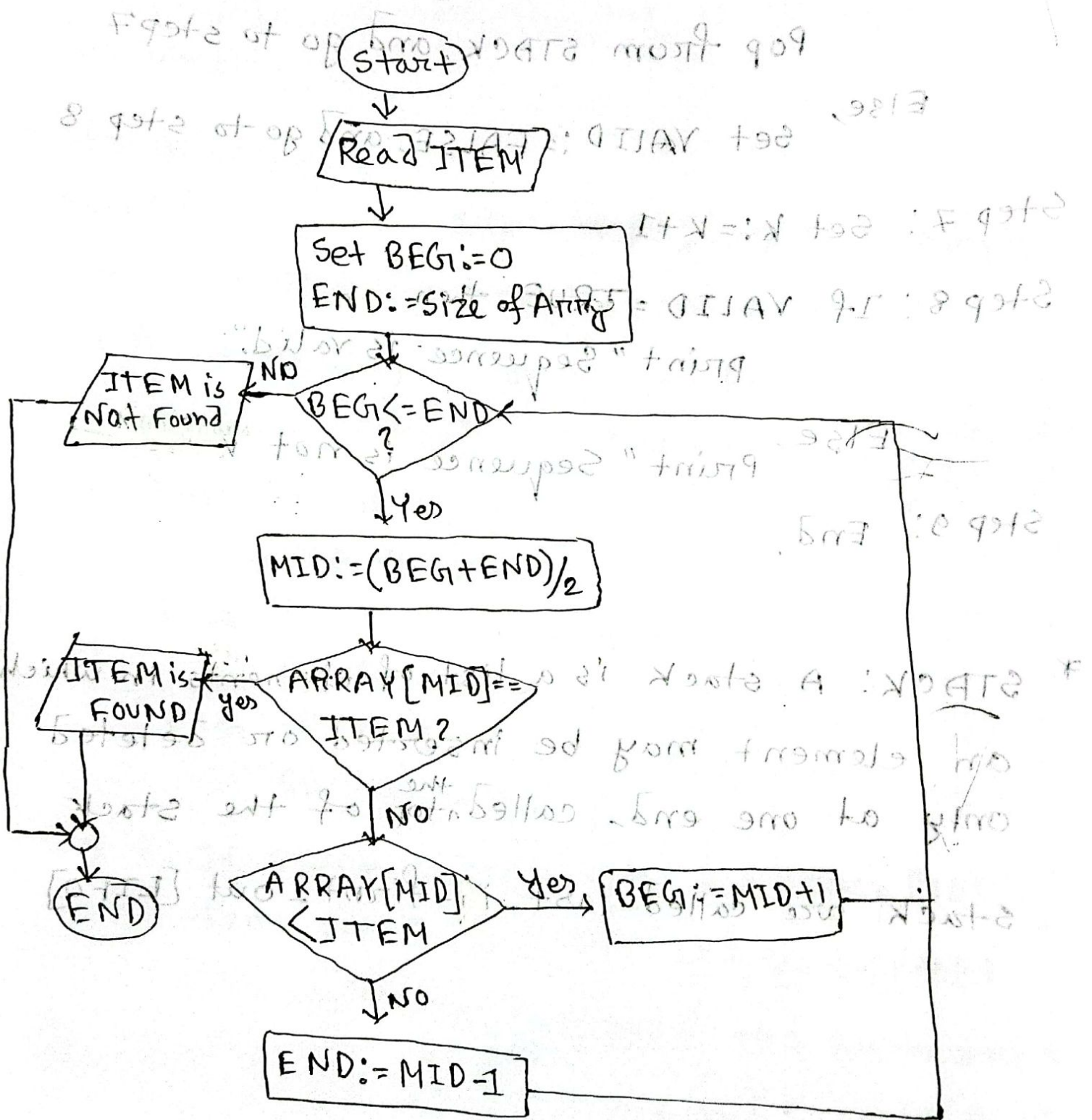
Step 8: If $VALID = TRUE$, then
print "Sequence is valid".

Else, print "Sequence is not valid".

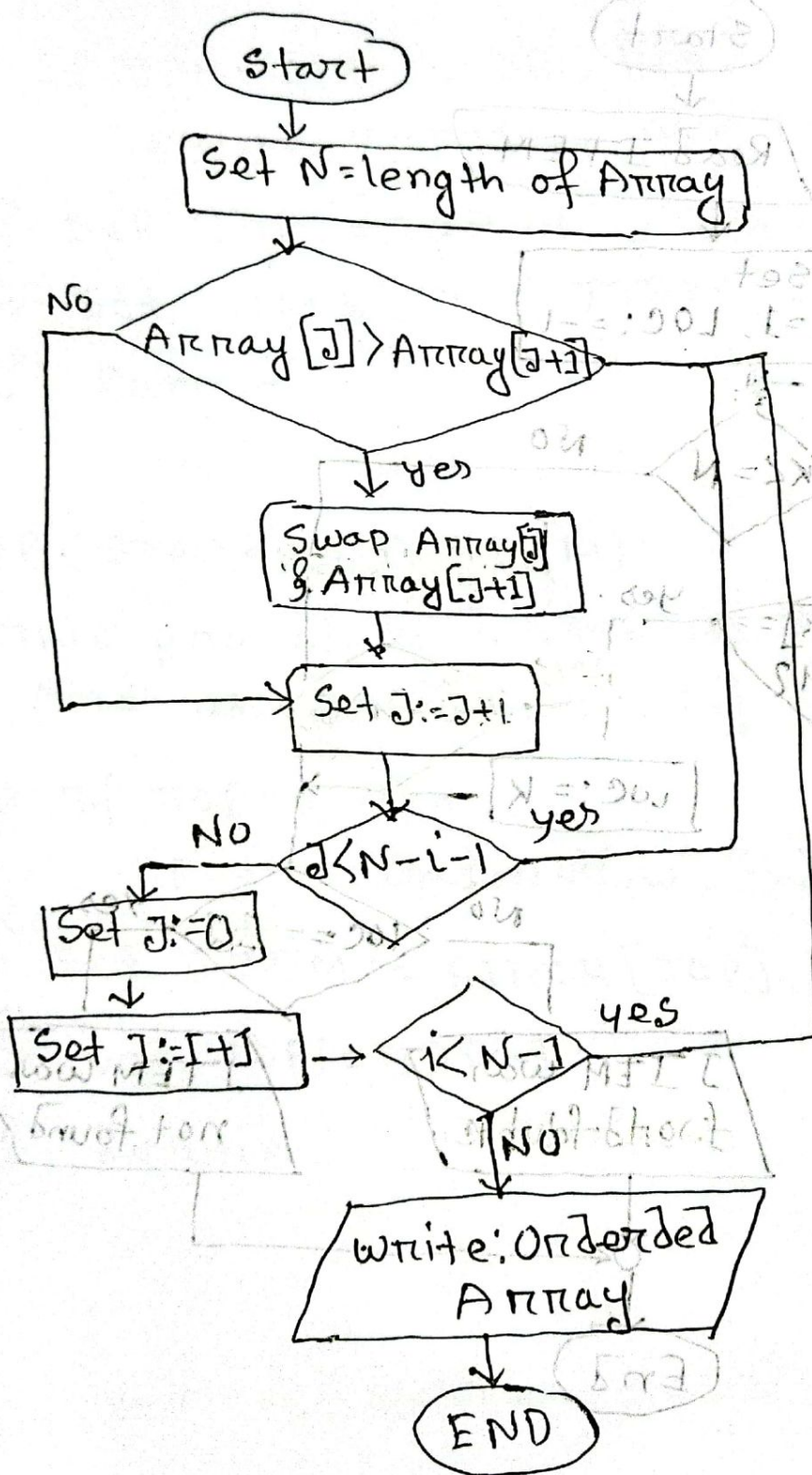
Step 9: End.

* **STACK**: A stack is a list of elements in which an element may be inserted or deleted only at one end, called ^{the} top of the stack. Stack are called last-in-first-out [LIFO].

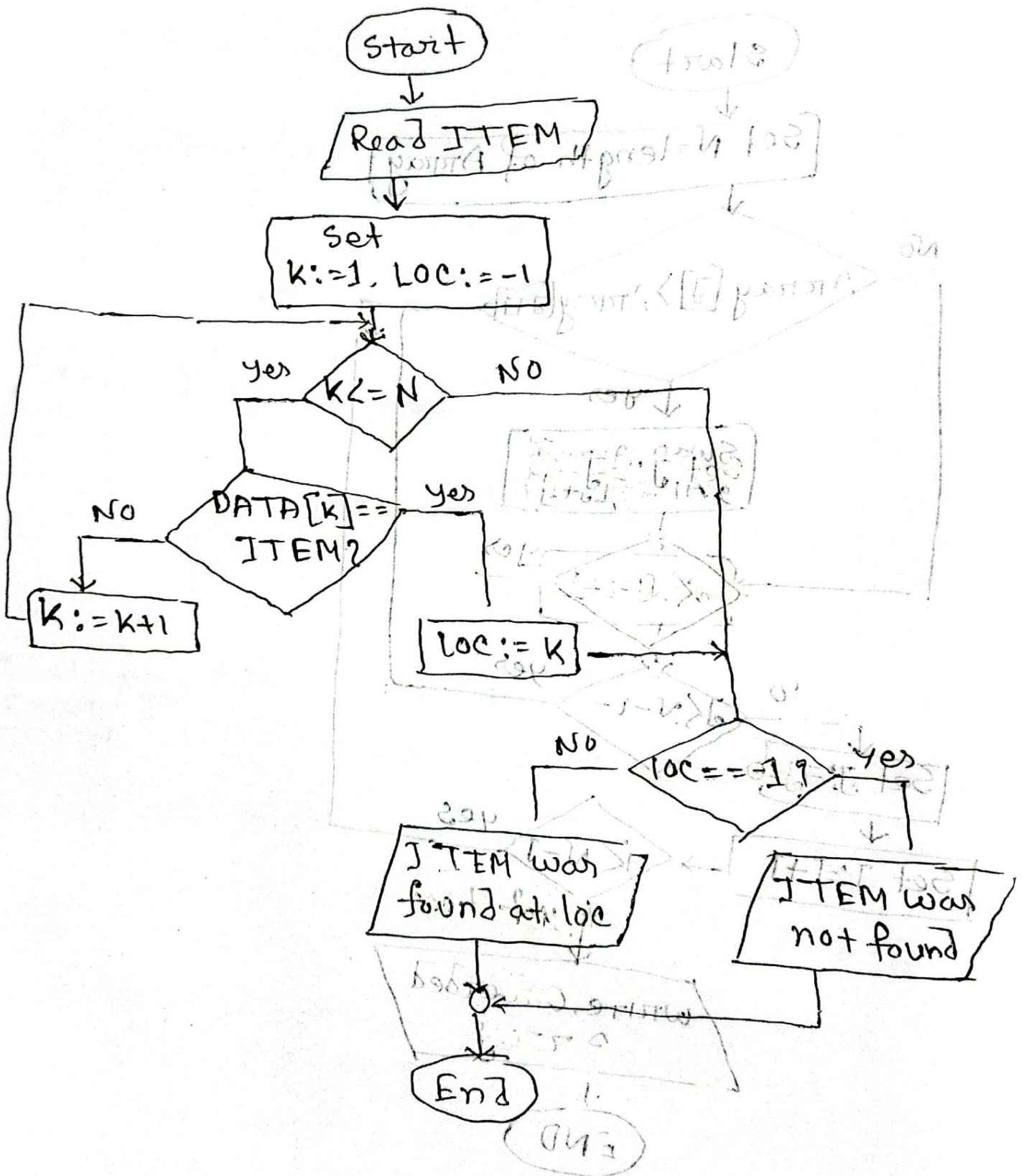
⑧ Binary Search Flow chart:



9 Bubble sort Flow chart



Linear chart Flowchart



⑥ PUSH (STACK, TOP, MAX_SIZE, ITEM):

This procedure pushes an ITEM onto the STACK

① If TOP = MAX_SIZE, then:

print overflow, and return

② Set TOP := TOP + 1

③ Set STACK[TOP] := ITEM

④ Return

POP (STACK, TOP, ITEM)

[This procedure deletes the TOP item from STACK

And assigns it to ITEM]

① If TOP = 0 then,

print underflow, and return

② Set ITEM := STACK[TOP]

③ Set TOP := TOP - 1

④ Return

DATA

STACK[0] = 10, STACK[1] = 20, STACK[2] = 30, STACK[3] = 40

⑧ Steps of binary search algorithm.

Given that

21	25	30	35	41	45	53	64	68	71	82	89	101
1	2	3	4	5	6	7	8	9	10	11	12	13

For, ITEM = 41

Pass-1: $BEG = 1$, $END = 13$, $MID = 7$

$DATA[7] = 53$

ITEM = 41

$ITEM < DATA[7]$

$END = MID - 1 = 6$

Pass-2: $BEG = 1$, $END = 6$, $MID = 3$

$DATA[3] = 30$

ITEM = 41

$ITEM > DATA[3]$

$BEG = MID + 1 = 4$

Pass-3: $BEG = 4$, $END = 6$, $MID = 5$

$DATA[5] = 41$

ITEM = 41

So, $\therefore$ ITEM = DATA[5]. So, ITEM was found

For, ITEM = 97

Pass 1:- $BEG = 1$, $END = 13$, $MID = 7$

$DATA[7] = 53$

$DATA[7] < ITEM$

$BEG = MID + 1 = 7 + 1 = 8$

Pass 2:- $BEG = 8$, $END = 13$, $MID = 10$

$DATA[10] = 71$

$DATA[10] < ITEM$

$BEG = MID + 1 = 10 + 1 = 11$

Pass 3:- $BEG = 11$, $END = 13$, $MID = 12$

$DATA[12] = 89$

$DATA[12] < ITEM$

$BEG = MID + 1 = 13$

Pass 4:-

$BEG = 13$, $END = 13$, $MID = 13$

$DATA[13] = 101$

$ITEM = 97$

$END = 13 - 1 = 12$

$BEG > END$

So, ITEM was not found